

Gabarito

Exercício 1: Par ou Ímpar

```
#include <stdio.h>
int main() {
    int n;
    scanf("%d", &n);
    if (n % 2 == 0) printf("Par\n");
    else printf("Impar\n");
    return 0;
}
```

Explicação detalhada:

Leitura via scanf. O operador % retorna o resto da divisão. Se $n \% 2 == 0$, o número é par.

Exercício 2: Soma e Média

```
#include <stdio.h>
int main() {
    int v[10], soma = 0;
    for (int i = 0; i < 10; i++) {
        scanf("%d", &v[i]);
        soma += v[i];
    }
    printf("Soma: %d\nMedia: %.2f\n", soma, soma / 10.0);
    return 0;
}
```

Explicação detalhada:

Uso de array estático. Loop acumula valores. Conversão para double garante precisão na média.

Exercício 3: Maior elemento

```
int maior(int v[], int n) {
    int max = v[0];
    for (int i = 1; i < n; i++)
        if (v[i] > max) max = v[i];
    return max;
}
```

Explicação detalhada:

Algoritmo linear $O(n)$. Mantém o maior valor encontrado até o momento.

Exercício 4: Troca com ponteiros

```
void troca(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

Explicação detalhada:

Uso de indireção (*). Permite modificar variáveis externas via endereço.

Exercício 5: Vetor dinâmico

```
#include <stdlib.h>  
#include <stdio.h>  
int main() {  
    int n;  
    scanf("%d", &n);  
    int *v = malloc(n * sizeof(int));  
    for (int i = 0; i < n; i++)  
        scanf("%d", &v[i]);  
    free(v);  
    return 0;  
}
```

Explicação detalhada:

malloc aloca memória no heap. Importante liberar com free para evitar leaks.

Exercício 6: Struct Processo

```
typedef struct {  
    int pid;  
    int tempo_execucao;  
} Processo;
```

Explicação detalhada:

Define tipo composto usado em simulação de escalonamento.

Exercício 7: Realloc

```
v = realloc(v, novo_tamanho * sizeof(int));
```

Explicação detalhada:

realloc pode mover o bloco de memória. Sempre verificar retorno != NULL em produção.



Exercício 8: Free

```
free(v);
```

Explicação detalhada:

Libera memória previamente alocada no heap.

Exercício 9: Escrita em arquivo

```
FILE *f = fopen("numeros.txt", "w");  
for (int i = 1; i <= 100; i++)  
    fprintf(f, "%d\n", i);  
fclose(f);
```

Explicação detalhada:

fopen abre arquivo. fprintf escreve formatado. fclose libera recursos do SO.

Exercício 10: Leitura e contagem

```
FILE *f = fopen("numeros.txt", "r");  
int n, pares = 0;  
while (fscanf(f, "%d", &n) != EOF)  
    if (n % 2 == 0) pares++;  
printf("Pares: %d\n", pares);  
fclose(f);
```

Explicação detalhada:

fscanf retorna EOF ao fim. Processamento linha a linha.

Exercício 11: Cópia de arquivo

```
FILE *f1 = fopen("origem.txt", "r");  
FILE *f2 = fopen("destino.txt", "w");  
char c;  
while ((c = fgetc(f1)) != EOF)  
    fputc(c, f2);  
fclose(f1);  
fclose(f2);
```

Explicação detalhada:

Operação em nível de byte. Simula utilitário cp simplificado.

Exercício 12: Leitura de processos

```
#include <stdio.h>  
typedef struct {
```



```
int pid;
int tempo_execucao;
} Processo;

int main() {
    FILE *f = fopen("processos.txt", "r");
    Processo p[100];
    int i = 0;
    while (fscanf(f, "%d %d", &p[i].pid, &p[i].tempo_execucao) != EOF)
        i++;
    fclose(f);
    return 0;
}
```

Explicação detalhada:

Carrega dados estruturados de arquivo para memória.

Exercício 13: fork

```
#include <unistd.h>
#include <stdio.h>
int main() {
    fork();
    printf("Executando\n");
    return 0;
}
```

Explicação detalhada:

fork duplica processo. Código após fork executa em ambos.

Exercício 14: Pai e filho

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();
    if (pid == 0)
        printf("Filho\n");
    else
        printf("Pai\n");
    return 0;
}
```

Explicação detalhada:

Valor de retorno do fork diferencia pai e filho.



Exercício 15: wait

```
#include <sys/wait.h>
wait(NULL);
```

Explicação detalhada:

Bloqueia até término do processo filho.

Exercício 16: exec

```
#include <unistd.h>
execl("/bin/ls", "ls", NULL);
```

Explicação detalhada:

Substitui imagem do processo atual por outro programa.

Exercício 17: FCFS

```
for (int i = 0; i < n; i++) {
    espera[i] = tempo_total;
    tempo_total += burst[i];
}
```

Explicação detalhada:

Escalonamento simples por ordem de chegada.

Exercício 18: SJF

```
// ordenar processos por tempo_execucao
```

Explicação detalhada:

Seleciona menor tempo primeiro. Minimiza tempo médio de espera.

Exercício 19: Round Robin

```
while (restante > 0) {
    for (int i = 0; i < n; i++) {
        if (burst[i] > 0) {
            int exec = (burst[i] > quantum) ? quantum : burst[i];
            burst[i] -= exec;
            tempo += exec;
        }
    }
}
```



Explicação detalhada:

Simula fatias de tempo com preempção.

Exercício 20: Tempo médio

```
tempo_medio = soma_espera / n;
```

Explicação detalhada:

Métrica fundamental para avaliar escalonadores.